

Đại học quốc gia TP.HCM
Trường đại học công nghệ thông tin



Ngành: Khoa học máy tính

Môn học: Cấu trúc dữ liệu và giải thuật

Tài liệu học tập

Sinh viên:

Trần Quang Trường - MSSV: 24521901

Mục lục

1	Tổng quan	3
1.1	Các khái niệm cơ bản	3
1.2	Vai trò của CTDL và Giải thuật	3
2	Tìm kiếm và Sắp xếp	3
2.1	Tìm kiếm	3
2.1.1	Tìm kiếm nhị phân (Binary Search)	4
2.1.2	Tìm kiếm nội suy (Interpolation Search)	4
2.1.3	Tìm kiếm nhảy (Jump Search)	4
2.1.4	Tìm kiếm theo cấp số nhân (Exponential search)	5
2.2	Sắp xếp	5
3	Bảng băm	8
3.1	Các khái niệm cơ bản	8
3.2	Các loại bảng băm	8
3.2.1	Hàm băm dùng phép chia	8
3.2.2	Hàm băm dùng phép nhân	8
3.2.3	Hàm băm phổ quát	8
3.3	Giải quyết đụng độ	9
3.3.1	Phương pháp nối kết (chaining)	9
3.3.2	Phương pháp địa chỉ mở (open address)	10
3.3.3	Băm lại(rehashing)	11
3.4	Độ phức tạp	12
4	Cryptography and applications	12
5	Đối sánh chuỗi ký tự	12
5.1	Bài toán	12
5.2	Các giải thuật tiêu biểu	12
5.2.1	Thuật toán KMP (Knuth–Morris–Pratt)	12
5.2.2	Thuật toán Boyer–Moore	14
5.2.3	Thuật toán Rabin–Karp	17
6	Cấu trúc dữ liệu động	19
6.1	Khái niệm, vai trò của CTDL động	19
6.2	Danh sách liên kết, các hình thức tổ chức danh sách	19
6.2.1	Danh sách liên kết đơn	20
6.2.2	Danh sách liên kết vòng	23
6.2.3	Danh sách liên kết kép	24
7	CTDL Trừu tượng	24
7.1	Queue (Hàng đợi)	24
7.1.1	Khi nào sử dụng queue?	25
7.1.2	Độ phức tạp của queue	25
7.1.3	Code minh họa queue bằng danh sách liên kết (C++)	25
7.2	Stack (Ngăn xếp)	26

7.2.1	Khi nào sử dụng stack?	26
7.2.2	Độ phức tạp của stack	27
7.2.3	Code minh họa stack bằng danh sách liên kết (C++)	27
7.3	Priority Queue (Hàng đợi ưu tiên)	28
7.3.1	Khi nào sử dụng priority queue?	28
7.3.2	Độ phức tạp của priority queue	28
7.3.3	Code minh họa priority queue bằng danh sách liên kết (C++)	28
8	CTDL cây	30
8.1	Khái niệm, các tính chất	30
8.2	Duyệt theo chiều sâu (Depth-First Traversal)	30
8.3	Duyệt theo chiều rộng (Breadth-First Traversal / Level-order)	31
8.4	Một số kiểu duyệt biến thể	31
8.5	Các loại cây thường gặp	31
8.5.1	BST	31
8.6	Các biến thể của BST	32
8.7	B-Tree	35
8.7.1	Các biến thể của B-Tree	35
8.8	Các thao tác cơ bản	35
8.8.1	Cách cài đặt	36
8.8.2	Các khái niệm cơ bản	37
8.8.3	Điểm nổi bật của B-Tree	37
8.8.4	Ứng dụng	37
9	Đồ thị	38
9.1	Các khái niệm cơ bản	38
9.2	Biểu diễn đồ thị trên máy tính	38
9.3	Các thuật toán duyệt đồ thị	39
9.3.1	BFS (Breadth-First Search)	39
9.3.2	DFS (Depth-First Search)	39
9.4	Bài toán đường đi	39
9.4.1	Dijkstra	39
9.4.2	Bellman-Ford	40

Tóm tắt nội dung

Nhằm để chuẩn bị thật tốt cho kỳ thi sắp tới, tài liệu được sinh ra với mục đích tổng hợp các kiến thức của môn học. Tài liệu chỉ mang tính chất tham khảo, vui lòng không copy toàn bộ nội dung. Xin nhắc lại, tác giả không chịu trách nhiệm cho nội dung của tài liệu. Ngoài ra, đây là **nguồn tài liệu đóng**, vui lòng **không chia sẻ** cho bất cứ ai dưới mọi hình thức nào nếu chưa có sự cho phép của tác giả.

Lời tựa

Ya hallo, mình là QioCas. Trước hết, mình xin chúc các bạn có một buổi thi thật bùng nổ! Mình rất hi vọng tài liệu mình có thể giúp ích cho các bạn. Tài liệu không thể tránh khỏi những sai sót vì thế nên các bạn cần chọn lọc các thông tin cẩn thận nhé. Ngoài ra, mình muốn gửi lời cảm ơn chân thành nhất tới Bùi Huỳnh Tây, Thức, Mai Quốc Anh, Phú Duy, Đạt, Gia Bảo, Khải Lạc, Quốc Phú những người đã giúp mình hoàn thành tài liệu này. Mình cảm ơn các bạn. Bên cạnh đó, mình không muốn public tài liệu này vì một phần do tài liệu không đảm bảo về độ chính xác. Hoặc chỉ đơn giản vì mình ích kỉ chỉ giữ nó cho bản thân? Ai biết được. Cuối cùng, mình muốn nói là:

"Bình tĩnh, tự tin, chiến thắng, mọi thứ sẽ ổn thôi!"

1 Tổng quan

1.1 Các khái niệm cơ bản

- Cấu trúc dữ liệu là một cách tổ chức và lưu trữ dữ liệu trong máy tính sao cho có thể truy cập và sửa đổi một cách hiệu quả.
- Giải thuật (hay thuật toán) là một tập hợp các bước hướng dẫn cụ thể, rõ ràng và có trình tự logic để giải quyết một vấn đề hoặc thực hiện một nhiệm vụ nào đó. Giải thuật có thể được áp dụng trong nhiều lĩnh vực khác nhau, từ toán học, khoa học máy tính đến cuộc sống hàng ngày.

1.2 Vai trò của CTDL và Giải thuật

- Vai trò của CTDL là để: Phản ánh được chính xác dữ liệu thực tế; Dễ dàng xử lý trên máy tính; Tiết kiệm tài nguyên hệ thống.
- Giải thuật được thiết kế để đáp ứng: Tính đúng đắn; Tính xác định; Tính dừng.

2 Tìm kiếm và Sắp xếp

2.1 Tìm kiếm

Ta có tìm kiếm tuyến tính (Sequently) và tìm kiếm theo đoạn (Interval). Ta có linear search thuộc tìm kiếm tuyến tính. Ta sẽ cùng tìm hiểu một số thuật toán tìm kiếm theo đoạn:

2.1.1 Tìm kiếm nhị phân (Binary Search)

Code C++ minh họa:

```
1 int binarySearch (int A[], int n, int x){
2     int l = 0, r = n-1;
3     while (l <= r) {
4         m = (l + r) / 2;
5         if (x == A[m]) return m;
6         if (x < A[m]) r = m - 1;
7         else l = m + 1;
8     }
9     return -1;
10 }
```

Phân tích độ phức tạp:

Ta xét trường hợp xấu nhất (worst case), số lần so sánh $C_{\text{worst}}(n)$ được mô tả bằng công thức sau:

$$C_{\text{worst}}(n) = C_{\text{worst}}\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 1 \quad \text{với } n > 1$$

$$\text{với } C_{\text{worst}}(1) = 1.$$

Với n bất kỳ, ta có độ phức tạp sau:

$$C_{\text{worst}}(n) = \lfloor \log_2 n \rfloor + 1 = \lceil \log_2(n+1) \rceil$$

- Độ phức tạp thời gian: $O(\log n)$
- Độ phức tạp không gian: $O(1)$

2.1.2 Tìm kiếm nội suy (Interpolation Search)

Ý tưởng: Thay vì $m = (l + r)/2$ như ở Binary Search, lúc này:

$$m = l + \frac{(r - l) \cdot (x - a_l)}{a_r - a_l}$$

Phân tích độ phức tạp:

- Độ phức tạp thời gian:
 - Trường hợp tốt nhất: phần tử cần tìm ở đúng vị được nội suy $\rightarrow$ số lần lặp là 1 $\rightarrow$ độ phức tạp hằng số $O(1)$
 - Trường hợp xấu nhất: giá trị khóa lớn nhất hoặc nhỏ nhất chênh lệch quá lớn so với giá trị kỳ vọng $\rightarrow$ tìm tuyến tính $\rightarrow$ độ phức tạp $O(n)$.
 - Trường hợp trung bình: độ phức tạp $O(\log(n))$.
- Độ phức tạp không gian: $O(1)$

2.1.3 Tìm kiếm nhảy (Jump Search)

Aka kĩ thuật chia căn.

2.1.4 Tìm kiếm theo cấp số nhân (Exponential search)

Code C++ minh họa:

```
1 int exponential_search(int a[], int size, int x)
2 {
3     if (size == 0) {
4         return -1;
5     }
6
7     int bound = 1;
8     while (bound < size && arr[bound] < x) {
9         bound *= 2;
10    }
11
12    // function binarySearch(int a[], int x, int l, int r)
13    return binarySearch(a, x, bound/2, min(bound, size));
14 }
```

Phân tích độ phức tạp:

- Độ phức tạp thời gian:
 - Trường hợp tốt nhất: $O(1)$
 - Trường hợp xấu nhất: $O(\log(n))$
 - Trường hợp trung bình: độ phức tạp $O(\log(i))$
- Độ phức tạp về không gian: $O(1)$

2.2 Sắp xếp

Trong tài liệu [Sorting Algorithms] - mình từng viết, mình đã phân loại và liệt kê các thuật toán. Do đó, mình sẽ nói những phần mà trong tài liệu trước mình chưa đề cập.

Thuật toán	Độ phức tạp	Bộ nhớ phụ	Ổn định
Interchange sort	$O(n^2)$	Không	Có
Bubble sort	$O(n^2)$	Không	Có
Quick sort	$O(n \log n)$	Đệ quy $O(\log n)$	Không
Shake sort	$O(n^2)$	Không	Có
Insertion sort	$O(n^2)$	Không	Có
Binary insertion sort	$O(n^2)$	Không	Có
Shell sort	Khoảng $O(n \log n)$	Không	Không
Selection sort	$O(n^2)$	Không	Không
Heap sort	$O(n \log n)$	Không	Không
Merge sort	$O(n \log n)$	$O(n)$	Có
Natural merge sort	$O(n \log n)$	$O(n)$	Có
K-way merge sort	$O(n \log k)$	$O(n) + heap$	Có
Counting sort	$O(n + k)$	$O(k)$	Có
Radix sort	$O(nk)$	$O(n + k)$	Có

Thuật toán	Ưu điểm	Nên dùng khi
Interchange sort	Rất đơn giản, dễ cài	Mảng nhỏ, demo, học thuật toán cơ bản
Bubble sort	Cực kỳ dễ viết, phát hiện mảng gần sắp xếp	Mảng rất nhỏ hoặc dữ liệu gần sắp xếp
Quick sort	Rất nhanh, ít bộ nhớ phụ, dùng nhiều thực tế	Dữ liệu lớn, cần tốc độ cao
Shake sort	Hoạt động tốt hơn bubble khi dữ liệu gần sắp xếp	Mảng nhỏ, gần sắp xếp
Insertion sort	Đơn giản, nhanh với mảng nhỏ hoặc gần sắp xếp	Mảng nhỏ, dữ liệu gần sắp xếp
Binary insertion sort	Ít phép so sánh hơn insertion sort	Mảng nhỏ, cần giảm phép so sánh
Shell sort	Nhanh hơn insertion sort, dễ cài	Mảng trung bình, không cần ổn định
Selection sort	Ít phép đổi chỗ, dễ cài	Mảng nhỏ, không yêu cầu tốc độ cao
Heap sort	Không cần bộ nhớ phụ, luôn $O(n \log n)$	Dữ liệu lớn, bộ nhớ hạn chế, không cần ổn định
Merge sort	Ổn định, tốt với dữ liệu lớn	Dữ liệu lớn, cần ổn định
Natural merge sort	Tốt hơn merge sort khi dữ liệu có sẵn chuỗi tăng	File lớn, dữ liệu có cấu trúc tăng tự nhiên
K-way merge sort	Merge nhiều file lớn hiệu quả	Sắp xếp file ngoài đĩa, xử lý file lớn
Counting sort	Rất nhanh với giá trị nhỏ	Số nguyên nhỏ, dữ liệu phân bố hẹp
Radix sort	Rất nhanh, không phụ thuộc so sánh	Số nguyên nhiều chữ số, cần tốc độ cao

- Offline Sort

- Định nghĩa:

- Thuật toán sắp xếp biết toàn bộ dữ liệu đầu vào trước khi bắt đầu sắp xếp.
- Chỉ bắt đầu sắp xếp sau khi nhận đủ tất cả phần tử.
- Tối ưu thứ tự toàn cục.

- Ưu điểm:

- Có thể sử dụng các kỹ thuật chia để trị, tối ưu bộ nhớ.
- Thường có độ phức tạp tốt hơn trong các bài toán sắp xếp hoàn chỉnh.

- Online Sort

- Định nghĩa: Thuật toán sắp xếp mà dữ liệu đến dần theo thời gian, và thuật toán phải duy trì thứ tự đã sắp xếp sau mỗi lần thêm phần tử mới.
- Không cần đợi đủ dữ liệu mới bắt đầu sắp xếp.

Thuật toán	Loại sắp xếp	Ghi chú
Counting Sort	Offline	Cần biết toàn bộ dãy trước để đếm và tính vị trí.
Selection Sort	Offline	Cần duyệt toàn bộ dãy để tìm phần tử nhỏ nhất cho từng vị trí.
Quick Sort	Offline	Chia để trị, cần toàn bộ dãy trước khi sắp xếp.
Merge Sort	Offline	Chia để trị, cần toàn bộ dãy trước khi sắp xếp.
Radix Sort	Offline	Cần toàn bộ dãy để xử lý các chữ số từ thấp đến cao hoặc ngược lại.
Heap Sort	Offline	Xây dựng Heap từ toàn bộ dãy trước khi trích xuất dần.
Bubble Sort	Offline	Cần biết toàn bộ dãy trước để thực hiện so sánh và hoán đổi nhiều lần.

Bảng 1: Phân loại các thuật toán sắp xếp: Online vs Offline

	TỐT NHẤT	TRUNG BÌNH	XẤU NHẤT
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Natural merge sort	$O(n)$	$O(n \log n)$	$O(n \log n)$
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Counting sort	$O(n + k)$	$O(n + k)$	$O(n + k)$
Radix sort	$O(m \cdot n)$	$O(m \cdot n)$	$O(m \cdot n)$

- Chia để trị: Merge sort, Quick sort, Heap sort, Binary Search
- Vét cạn: Selection sort, Insertion sort, Bubble sort, Linear Search
- Tham lam: Counting sort, Radix sort, Dijkstra, Prim, Kruskal

	TỐT NHẤT	TRUNG BÌNH	XẤU NHẤT
Linear search (tuyến tính)	$O(1)$	$O(n)$	$O(n)$
Binary search (nhị phân)	$O(1)$	$O(\log n)$	$O(\log n)$
Interpolation search (nội suy)	$O(1)$	$O(\log \log n)$	$O(\log n)$

3 Bảng băm

3.1 Các khái niệm cơ bản

- **Khóa** là thuộc tính được dùng để xác định một phần tử.
- **Bảng băm (hash table)** là một cấu trúc dữ liệu có đặc điểm sau:
 - * Có kích thước m .
 - * Cho phép truy xuất ngẫu nhiên từng phần tử theo giá trị khóa.
 - * Độ phức tạp thời gian có thể đạt $O(1)$ khi truy xuất phần tử.
- **Hàm băm** h ánh xạ khóa k của phần tử cần lưu trữ vào vị trí $h(k)$ trong bảng băm có kích thước m :

$$h : KEYS \rightarrow \{0, 1, \dots, m - 1\}$$

- **Đụng độ (collision)** là trường hợp hai giá trị khoá được ánh xạ vào cùng một giá trị chỉ số mảng.
- **Hệ số tải (load factor)** của bảng băm T có kích thước m khi lưu trữ n phần tử là $\lambda = n/m$.

3.2 Các loại bảng băm

3.2.1 Hàm băm dùng phép chia

- $h(k) = k \bmod m$.
- Khi chọn m nên là **một số nguyên tố** và né các số lũy thừa của 2.

3.2.2 Hàm băm dùng phép nhân

- $h(k) = \lfloor m * (k * A \bmod 1) \rfloor$
- $A \in (0, 1)$.
- Khi chọn m nên là một số có dạng $2^p, p \in \mathbb{N}^*$.
- Theo Knuth, $A = (\text{sqrt}(5) - 1)/2$.

3.2.3 Hàm băm phổ quát

- $H = \{H_{a,b}(k) = ((a * k + b) \bmod p) \bmod m\}$.
- a, b là hai số tự nhiên tùy ý, $1 \leq a \leq p - 1, 0 \leq b \leq p - 1$.
- p là số nguyên tố lớn hơn tất cả các giá trị khóa.

3.3 Giải quyết đụng độ

3.3.1 Phương pháp nối kết (chaining)

- Các phần tử đụng độ tại giá trị băm i được kết nối với nhau thành một danh sách liên kết thứ i trong bảng băm T .
- Một ứng dụng của phương pháp này là `std::unordered_map` của C++. Việc sử dụng `std::unordered_map` rất tiện lợi vì không yêu cầu cài đặt phức tạp. Tuy nhiên, nhược điểm của nó là khó tùy chỉnh hoặc định nghĩa lại hàm băm trong một số trường hợp cụ thể khi cần kiểm soát sâu hơn quá trình băm.
- Nếu bộ nhớ không quá hạn chế và muốn code đơn giản, hiệu quả ổn định, ta có thể sử dụng phương pháp Chaining.
- Phương pháp Chaining sẽ không đạt hiệu quả tốt khi ta truy cập quá nhiều vào cùng một ô trong bảng băm.

```
1  const int M = 23;
2  struct Node {
3      Node* next = nullptr;
4      int key, counter;
5      Node(int x): key(x), counter(1) {}
6  };
7
8  using Pnode = Node*;
9
10 struct LinkedList {
11     Pnode head = nullptr, tail = nullptr;
12 } Table[M];
13
14 void insert(int k) {
15     int hashkey = k % M;
16     bool found = 0;
17     for(Pnode p = Table[hashkey].head; p != nullptr; p = p->next) {
18         if(p->key == k) {
19             p->counter++;
20             found = 1;
21             break;
22         }
23     }
24
25     if(found == 0) {
26         Pnode pointerkey = new Node(k);
27         if(Table[hashkey].head == nullptr) {
28             Table[hashkey].head = Table[hashkey].tail = pointerkey;
29         } else {
30             Table[hashkey].tail->next = pointerkey;
31             Table[hashkey].tail = pointerkey;
32         }
33     }
34 }
35
36
37 int count(int k) {
38     int hashkey = k % M;
```

```

39     for(Pnode p = Table[hashkey].head; p != nullptr; p = p->next) {
40         if(p->key == k) {
41             return p->counter;
42         }
43     }
44     return 0;
45 }
46
47 void erase(int k) {
48     int hashkey = k % M;
49     for(Pnode prev = nullptr, p = Table[hashkey].head; p != nullptr; prev
50         = p, p = p->next) {
51         if(p->key == k) {
52             p->counter--;
53             if(p->counter == 0) {
54                 if(prev == nullptr) {
55                     Table[hashkey].head = p->next;
56                     if(p == Table[hashkey].tail) Table[hashkey].tail =
57                         nullptr;
58                     delete p;
59                 } else {
60                     prev->next = p->next;
61                     if(p == Table[hashkey].tail) Table[hashkey].tail =
62                         prev;
63                     delete p;
64                     break;
65                 }
66             }
67         }
68     }
69     return;
70 }
71 }

```

3.3.2 Phương pháp địa chỉ mở (open address)

- Phương pháp địa chỉ mở (open address): nếu giá trị băm i trên bảng băm T đã tồn tại phần tử khác thì thăm dò vị trí phù hợp.
 - * Công thức: $h_i(k) = (h(k) + f(i)) \bmod m$
 - * Thăm dò tuyến tính (linear probing): $f(i) = i$
 - * Thăm dò bậc hai (quadratic probing): $f(i) = a * i^2 + b * i$
 - * Băm kép (double hashing): $f(i) = i * h'(k)$, $h'(k) = R - (k \bmod R)$. R là số nguyên tố hơn m . $h'(k)$ không được bằng 0.
- Nếu yêu cầu cần tối ưu bộ nhớ và tốc độ truy cập, Băm kép (double hashing) là lựa chọn tốt nhất. Vì nó sử dụng hai hash giúp giảm clustering hiệu quả cao. Tuy vậy, phương pháp này đòi hỏi phải hiểu rõ bảng băm và cài đặt cũng tương đối khó khăn.

```

1  const int M = 23;
2  const int EMPTY = -1;
3  const int DELETED = -2;
4
5  int Table[M];

```

```

6
7 // First hash function:  $h_1(k) = k \bmod M$ 
8 int h1(int key) {
9     return key % M;
10 }
11
12 // Second hash function:  $h_2(k) = R - (k \bmod R)$ ,  $R < M$  and should be prime
13 int h2(int key) {
14     const int R = 7;
15     return R - (key % R);
16 }
17
18 // Combined hash function:  $h(k, i) = (h_1(k) + i * h_2(k)) \% M$ 
19 int doubleHash(int key, int i) {
20     return (h1(key) + i * h2(key)) % M;
21 }
22
23 void insert(int key) {
24     for (int i = 0; i < M; i++) {
25         int idx = doubleHash(key, i);
26         if (Table[idx] == EMPTY || Table[idx] == DELETED) {
27             Table[idx] = key;
28             return;
29         }
30     }
31 }
32
33 bool search(int key) {
34     for (int i = 0; i < M; i++) {
35         int idx = doubleHash(key, i);
36         if (Table[idx] == EMPTY) return false;
37         if (Table[idx] == key) return true;
38     }
39     return false;
40 }
41
42 void erase(int key) {
43     if (search(key) == 0) return;
44     for (int i = 0; i < M; i++) {
45         int idx = doubleHash(key, i);
46         if (Table[idx] == key) {
47             Table[idx] = DELETED;
48             return;
49         }
50     }
51 }

```

3.3.3 Băm lại(rehashing)

– Băm lại (rehashing) cần được thực hiện khi:

- * Không thể thực hiện một thao tác thêm phần tử mới bất kỳ.
- * Bảng băm có hệ số tải đạt một giá trị xác định ($\lambda \geq 0.5$) làm độ phức tạp thời gian của thao tác tìm kiếm tăng.

- Ý tưởng của rehashing đơn giản là tạo một bảng băm mới, xong nhét lại các phần tử của bảng cũ qua bảng mới theo hàm băm mới.

3.4 Độ phức tạp

- Trong các trường hợp bình thường, độ phức tạp cho các thao tác Tìm kiếm, Thêm, Xóa hầu hết đạt độ phức tạp $O(1)$.
- Tuy nhiên, trong trường hợp tệ nhất, độ phức tạp cho các thao tác Tìm kiếm, Thêm, Xóa có thể lên tới $O(n)$.

4 Cryptography and applications

Mật mã học và Ứng dụng. Các kiến thức liên quan của chương này sẽ nằm trong tài liệu [Cryptography and applications].

5 Đối sánh chuỗi ký tự

5.1 Bài toán

- **Đối sánh mẫu (Pattern Matching)**: Kỹ thuật kiểm tra sự hiện diện của một mẫu trong một chuỗi ký tự đã cho.
- **Tìm kiếm chuỗi con (Substring Search)**: Bài toán tìm một hoặc nhiều lần xuất hiện của một chuỗi mẫu (độ dài M) trong một chuỗi văn bản lớn (độ dài N).
- Biểu thức chính quy (Regular Expression - Regex) như một công cụ mạnh mẽ để mô tả các mẫu.

5.2 Các giải thuật tiêu biểu

5.2.1 Thuật toán KMP (Knuth–Morris–Pratt)

1. Mục tiêu

Tìm tất cả các vị trí xuất hiện của chuỗi mẫu (pattern) trong chuỗi văn bản (text) một cách hiệu quả, tránh lặp lại việc so sánh.

2. Ý tưởng thuật toán

- Khi có **mismatch**, không quay lại từ đầu pattern như thuật toán brute-force.
- Sử dụng mảng `lps[]` (*longest proper prefix which is also suffix*) để biết phải nhảy tới đâu trong pattern.
- Tránh việc so sánh lại các ký tự đã trùng trước đó.

3. Tính mảng LPS

```

1 vector<int> computeLPS(string pattern) {
2     int m = pattern.size();
3     vector<int> lps(m, 0);
4     int len = 0;
5
6     for (int i = 1; i < m; ++i) {
7         while (len > 0 && pattern[i] != pattern[len])
8             len = lps[len - 1];
9
10        if (pattern[i] == pattern[len])
11            ++len;
12
13        lps[i] = len;
14    }
15    return lps;
16 }

```

Listing 1: Hàm tính mảng LPS

4. Thuật toán KMP

```

1 vector<int> KMP(string text, string pattern) {
2     vector<int> res;
3     int n = text.size(), m = pattern.size();
4     vector<int> lps = computeLPS(pattern);
5
6     int i = 0, j = 0;
7     while (i < n) {
8         if (text[i] == pattern[j]) {
9             ++i; ++j;
10        }
11
12        if (j == m) {
13            res.push_back(i - j);
14            j = lps[j - 1];
15        } else if (i < n && text[i] != pattern[j]) {
16            if (j > 0)
17                j = lps[j - 1];
18            else
19                ++i;
20        }
21    }
22    return res;
23 }

```

Listing 2: Thuật toán KMP tìm chuỗi con

5. Ưu và Nhược điểm

– Ưu điểm:

- * Thời gian chạy $O(N + M)$.
- * Không so lại các ký tự đã khớp.
- * Tìm được tất cả vị trí khớp trong văn bản.

– Nhược điểm:

- * Cần thêm mảng `lps` và tính trước.
- * Phức tạp hơn so với brute-force.

6. Khi nào nên dùng KMP?

- Khi chuỗi văn bản dài và cần nhiều lần tìm kiếm.
- Khi pattern có tính lặp lại rõ rệt.
- Khi yêu cầu độ phức tạp tối ưu và ổn định.

5.2.2 Thuật toán Boyer–Moore

1. Mục tiêu

Tìm kiếm chuỗi mẫu (pattern) trong văn bản (text) với hiệu năng cao, bằng cách bỏ qua nhiều ký tự không cần thiết khi xảy ra mismatch.

2. Ý tưởng thuật toán

- Duyệt pattern từ **phải sang trái**.
- Khi có mismatch, sử dụng hai chiến lược để dịch pattern:
 - * **Bad Character Rule (Ký tự xấu)**.
 - * **Good Suffix Rule (Hậu tố tốt)**.
- Ở đây, ta cài đặt thuật toán dùng **Bad Character Rule** để đơn giản.

3. Bad Character Rule

- Nếu ký tự tại vị trí bị mismatch không khớp:
 - * Nếu ký tự đó có trong pattern → dịch pattern sao cho nó trùng lần xuất hiện gần nhất trong pattern.
 - * Nếu không có trong pattern → dịch pattern sang phải hoàn toàn qua ký tự đó.
- Bảng bad character lưu vị trí xuất hiện cuối cùng của mỗi ký tự trong pattern.

4. Cài đặt Boyer–Moore với Bad Character Rule

```

1 vector<int> buildBadCharTable(string pattern) {
2     vector<int> badChar(256, -1); // 256 characters ASCII
3     for (int i = 0; i < pattern.size(); ++i)
4         badChar[(unsigned char)pattern[i]] = i;
5     return badChar;
6 }
```

Listing 3: Tạo bảng bad character

```

1 int boyerMooreSearch(string text, string pattern) {
2     int n = text.size(), m = pattern.size();
3     if (m == 0) return 0;
4
5     vector<int> badChar = buildBadCharTable(pattern);
6     int shift = 0;
```

```

7
8   while (shift <= n - m) {
9       int j = m - 1;
10
11       while (j >= 0 && pattern[j] == text[shift + j])
12           --j;
13
14       if (j < 0) {
15           return shift; // found at pos shift
16       } else {
17           char mismatched = text[shift + j];
18           int last = badChar[(unsigned char)mismatched];
19           shift += max(1, j - last);
20       }
21   }
22
23   return -1; // not found
24 }

```

Listing 4: Thuật toán Boyer–Moore cơ bản

4. Good Suffix Rule (Hậu tố tốt)

- Khi có một phần hậu tố trong pattern đã khớp trước khi mismatch:
 - * Nếu phần hậu tố đó xuất hiện ở vị trí khác trong pattern, dịch pattern sao cho chúng trùng.
 - * Nếu không xuất hiện lại, dịch pattern sao cho hậu tố khớp với tiền tố (prefix) của pattern.
- Mảng `goodSuffixShift[]` dùng để lưu khoảng dịch phù hợp cho từng vị trí mismatch.

```

1 vector<int> buildBadCharTable(const string& pattern) {
2     vector<int> badChar(256, -1);
3     for (int i = 0; i < pattern.size(); ++i)
4         badChar[(unsigned char)pattern[i]] = i;
5     return badChar;
6 }

```

Listing 5: Hàm tạo bảng Bad Character

```

1 vector<int> buildGoodSuffixTable(const string& pattern) {
2     int m = pattern.length();
3     vector<int> shift(m + 1, m);
4     vector<int> border(m + 1);
5     int i = m, j = m + 1;
6     border[i] = j;
7
8     while (i > 0) {
9         while (j <= m && pattern[i - 1] != pattern[j - 1]) {
10             if (shift[j] == m)
11                 shift[j] = j - i;
12             j = border[j];
13         }
14         --i; --j;
15         border[i] = j;
16     }

```



```

17
18     for (int i = 0; i <= m; ++i)
19         if (shift[i] == m)
20             shift[i] = j;
21
22     return shift;
23 }

```

Listing 6: Hàm tạo bảng Good Suffix

```

1 void boyerMooreSearch(const string& text, const string& pattern) {
2     int n = text.size(), m = pattern.size();
3     if (m == 0) return;
4
5     vector<int> badChar = buildBadCharTable(pattern);
6     vector<int> goodSuffix = buildGoodSuffixTable(pattern);
7
8     int shift = 0;
9     while (shift <= n - m) {
10         int j = m - 1;
11
12         while (j >= 0 && pattern[j] == text[shift + j])
13             --j;
14
15         if (j < 0) {
16             cout << "Pattern appear at pos " << shift << endl;
17             shift += goodSuffix[0];
18         } else {
19             int badShift = j - badChar[(unsigned char)text[shift + j]];
20             int goodShift = goodSuffix[j + 1];
21             shift += max(1, max(badShift, goodShift));
22         }
23     }
24 }

```

Listing 7: Thuật toán Boyer–Moore hoàn chỉnh

5. Ưu và Nhược điểm

– Ưu điểm:

- * Có thể bỏ qua nhiều ký tự một lúc.
- * Rất nhanh trong thực tế, đặc biệt khi pattern ngắn.

– Nhược điểm:

- * Cài đặt đầy đủ khá phức tạp (nếu dùng cả Good Suffix Rule).
- * Không ổn định khi pattern có tính lặp cao.

6. Khi nào nên dùng Boyer–Moore?

- Khi văn bản rất dài, và pattern tương đối ngắn.
- Khi cần tốc độ thực tế cao.
- Khi muốn bỏ qua nhiều ký tự không cần so sánh.

5.2.3 Thuật toán Rabin–Karp

1. Mục tiêu

Tìm kiếm chuỗi mẫu (pattern) trong văn bản (text) bằng cách sử dụng hàm băm (hash) để so sánh chuỗi một cách hiệu quả.

2. Ý tưởng thuật toán

- Tính **giá trị băm** của pattern và của các đoạn con trong text có độ dài bằng pattern.
- So sánh giá trị băm. Nếu giống nhau, kiểm tra từng ký tự để xác nhận (vì có thể xảy ra va chạm băm – hash collision).
- Sử dụng kỹ thuật **rolling hash** để tính băm đoạn tiếp theo từ đoạn trước đó một cách nhanh chóng.

3. Công thức rolling hash

Với bảng chữ cái kích thước d (ví dụ $d = 256$ với ASCII), và một số nguyên tố lớn q :

$$\text{hash}(S) = (S_0 \cdot d^{m-1} + S_1 \cdot d^{m-2} + \dots + S_{m-1}) \mod q$$

Khi dịch sang đoạn tiếp theo, chỉ cần:

$$\text{new_hash} = (d \cdot (\text{old_hash} - S_i \cdot h) + S_{i+m}) \mod q$$

Với $h = d^{m-1} \mod q$

4. Cài đặt thuật toán Rabin–Karp

```
1 void rabinKarp(string text, string pattern, int d = 256, int q = 101) {
2     int n = text.size(), m = pattern.size();
3     int p = 0, t = 0, h = 1;
4
5     // Cal h = d^(m-1) % q
6     for (int i = 0; i < m - 1; ++i)
7         h = (h * d) % q;
8
9     // Cal hash of pattern and the first text's sequence.
10    for (int i = 0; i < m; ++i) {
11        p = (d * p + pattern[i]) % q;
12        t = (d * t + text[i]) % q;
13    }
14
15    for (int i = 0; i <= n - m; ++i) {
16        if (p == t) {
17            bool match = true;
18            for (int j = 0; j < m; ++j)
19                if (text[i + j] != pattern[j]) {
20                    match = false;
21                    break;
22                }
23            if (match)
24                cout << "Pattern appears at pos" << i << endl;
25        }
26    }
```

```

26
27     if (i < n - m) {
28         t = (d * (t - text[i] * h) + text[i + m]) % q;
29         if (t < 0) t += q;
30     }
31 }
32 }

```

Listing 8: Thuật toán Rabin–Karp

5. Ưu và Nhược điểm

– Ưu điểm:

- * Dễ cài đặt.
- * Dùng hash nên xử lý nhanh chuỗi dài.
- * Có thể tìm nhiều pattern cùng lúc nếu dùng kỹ thuật nâng cao.

– Nhược điểm:

- * Có thể xảy ra va chạm băm (hash collision).
- * Trường hợp xấu cần so từng ký tự → gần giống brute-force.

6. Khi nào nên dùng Rabin–Karp?

- Khi cần kiểm tra nhiều chuỗi mẫu trong cùng một văn bản.
- Khi chuỗi mẫu ngắn, văn bản dài, và muốn tối ưu bằng rolling hash.
- Khi không cần quá chắc chắn (cho phép sai số nhỏ, ví dụ trong kiểm tra trùng lặp).

So sánh các thuật toán tìm chuỗi

Tiêu chí	KMP	Boyer–Moore	Rabin–Karp
Chiến lược chính	Dò trái → phải, dùng LPS	Dò phải → trái, nhảy thông minh	Dùng hàm băm (hash) để so sánh
Tiền xử lý	Tính mảng LPS	Bảng bad char, good suffix	Tính hash ban đầu
Độ phức tạp thời gian	$O(N + M)$	Tốt: $O(N/M)$, Xấu: $O(NM)^*$	Trung bình: $O(N + M)$, Tệ: $O(NM)$
Không gian phụ	$O(M)$	$O(M)$	$O(1)$
Tìm tất cả vị trí	Dễ dàng	Có thể	Có thể
Hiệu quả thực tế	Ổn định, nhanh	Cực nhanh nếu pattern ngắn	Hiệu quả khi kiểm tra nhiều chuỗi
Pattern có lặp	Rất tốt	Không hiệu quả	Không ảnh hưởng nhiều
Hỗ trợ nhiều pattern	Không	Không	Có
Xác suất sai lệch	Không	Không	Có thể có va chạm băm
Dễ cài đặt	Vừa phải	Phức tạp (nếu dùng good suffix)	Đơn giản
Khi nên dùng	So khớp ổn định, text dài	Cần tốc độ cao, pattern ngắn	Kiểm tra nhiều chuỗi con

**Boyer–Moore có thể bị $O(NM)$ trong trường hợp tệ nhất nếu pattern và text bị thiết kế đặc biệt. Tuy nhiên, thực tế rất hiếm gặp.*

6 Cấu trúc dữ liệu động

6.1 Khái niệm, vai trò của CTDL động

- **Cấu trúc dữ liệu động (dynamic data structure)** là một loại cấu trúc dữ liệu cho phép thay đổi kích thước và hình dạng trong quá trình thực thi chương trình, trái ngược với cấu trúc dữ liệu tĩnh thường có kích thước cố định.
- Cấu trúc dữ liệu động đóng vai trò quan trọng trong việc xây dựng các chương trình máy tính linh hoạt và hiệu quả. Chúng cho phép lưu trữ và quản lý dữ liệu thay đổi kích thước trong quá trình thực thi, thay vì phải xác định kích thước cố định trước. Điều này giúp tối ưu hóa việc sử dụng bộ nhớ, tăng tốc độ xử lý và cải thiện khả năng mở rộng của các ứng dụng.

6.2 Danh sách liên kết, các hình thức tổ chức danh sách

- Danh sách liên kết (linked list) là một cấu trúc dữ liệu tuyến tính, trong đó các phần tử (node) được tổ chức thành một chuỗi, mỗi phần tử chứa dữ liệu và một con trỏ (hoặc liên kết) trỏ đến phần tử tiếp theo trong danh sách.

- Cấu trúc của mỗi phần tử (node):
 - * **Dữ liệu (data)**: Lưu giá trị thực (ví dụ: số nguyên, chuỗi ký tự, đối tượng, v.v.).
 - * **Liên kết (con trỏ)**: Lưu địa chỉ trỏ đến phần tử tiếp theo (và/hoặc phần tử trước, tùy loại danh sách).
- Danh sách liên kết tốn bộ nhớ hơn mảng vì:
 - * Ngoài lưu trữ dữ liệu, mỗi node cần lưu thêm địa chỉ (con trỏ), thường chiếm 4 bytes (trên hệ thống 32-bit) hoặc 8 bytes (trên hệ thống 64-bit) cho mỗi liên kết.
 - * Nếu là danh sách liên kết đôi, mỗi node lưu 2 con trỏ: **next** và **prev**.
- Ưu điểm:
 - * Linh hoạt, dễ thay đổi cấu trúc và kích thước khi chương trình đang chạy.
 - * Thích hợp cho dữ liệu biến động nhiều: thêm, xóa, hợp, tách, tái cấu trúc mà không cần cấp phát lại toàn bộ như mảng.
 - * Hỗ trợ các thao tác như nối, tách danh sách một cách tự nhiên.
- Nhược điểm:
 - * Truy xuất ngẫu nhiên chậm: muốn truy cập phần tử thứ k phải duyệt tuần tự từ đầu $\rightarrow$ thời gian $O(n)$.
 - * Tìm kiếm tuần tự, không hỗ trợ tìm kiếm nhảy nhanh như mảng (không dùng được tìm kiếm nhị phân trực tiếp).
 - * Không hỗ trợ truy cập chỉ số trực tiếp như `array[i]`.
- Có ba dạng danh sách liên kết chính: đơn, đôi và vòng.

6.2.1 Danh sách liên kết đơn

```

1 #include <iostream>
2 using namespace std;
3 struct Node {
4     int data;
5     Node* next;
6     Node(int val) : data(val), next(nullptr) {}
7 };
8 class SinglyLinkedList {
9 private:
10     Node* head;
11 public:
12     SinglyLinkedList() : head(nullptr) {}
13     bool isEmpty() { return head == nullptr; }
14     void addFront(int val) {
15         Node* newNode = new Node(val);
16         newNode->next = head;
17         head = newNode;
18     }
19     void addBack(int val) {
20         Node* newNode = new Node(val);
21         if (isEmpty()) {
22             head = newNode;
23             return;
24         }

```

```

25     Node* temp = head;
26     while (temp->next) temp = temp->next;
27     temp->next = newNode;
28 }
29 void insertAt(int pos, int val) {
30     if (pos == 0) {
31         addFront(val);
32         return;
33     }
34     Node* newNode = new Node(val);
35     Node* temp = head;
36     for (int i=0; temp && i<pos-1; i++)
37         temp = temp->next;
38     if (!temp) return; // non-existent
39     newNode->next = temp->next;
40     temp->next = newNode;
41 }
42 void removeFront() {
43     if (isEmpty()) return;
44     Node* temp = head;
45     head = head->next;
46     delete temp;
47 }
48 void removeBack() {
49     if (isEmpty()) return;
50     if (!head->next) {
51         delete head;
52         head = nullptr;
53         return;
54     }
55     Node* temp = head;
56     while (temp->next->next) temp = temp->next;
57     delete temp->next;
58     temp->next = nullptr;
59 }
60 void removeAt(int pos) {
61     if (pos == 0) {
62         removeFront();
63         return;
64     }
65     Node* temp = head;
66     for (int i=0; temp && i<pos-1; i++)
67         temp = temp->next;
68     if (!temp || !temp->next) return;
69     Node* del = temp->next;
70     temp->next = del->next;
71     delete del;
72 }
73 void display() {
74     Node* temp = head;
75     while (temp) {
76         cout << temp->data << " -> ";
77         temp = temp->next;
78     }
79     cout << "NULL\n";
80 }

```

```

81     Node* search(int val) {
82         Node* temp = head;
83         while (temp) {
84             if (temp->data == val) return temp;
85             temp = temp->next;
86         }
87         return nullptr;
88     }
89     void clear() {
90         while (!isEmpty()) removeFront();
91     }
92     void concat(SinglyLinkedList& other) {
93         if (isEmpty()) {
94             head = other.head;
95         } else {
96             Node* temp = head;
97             while (temp->next) temp = temp->next;
98             temp->next = other.head;
99         }
100         other.head = nullptr;
101     }
102     // MergeSort:
103     Node* mergeSort(Node* node) {
104         if (!node || !node->next) return node;
105         Node* mid = getMid(node);
106         Node* left = mergeSort(node);
107         Node* right = mergeSort(mid);
108         return merge(left, right);
109     }
110     void sort() {
111         head = mergeSort(head);
112     }
113 private:
114     Node* getMid(Node* node) {
115         Node* slow = node, *fast = node, *prev = nullptr;
116         while (fast && fast->next) {
117             prev = slow;
118             slow = slow->next;
119             fast = fast->next->next;
120         }
121         if (prev) prev->next = nullptr;
122         return slow;
123     }
124     Node* merge(Node* l1, Node* l2) {
125         Node dummy(0);
126         Node* tail = &dummy;
127         while (l1 && l2) {
128             if (l1->data < l2->data) {
129                 tail->next = l1;
130                 l1 = l1->next;
131             } else {
132                 tail->next = l2;
133                 l2 = l2->next;
134             }
135             tail = tail->next;
136         }

```

```

137         tail->next = (l1 ? l1 : l2);
138         return dummy.next;
139     }
140 };

```

6.2.2 Danh sách liên kết vòng

```

1
2 class CircularLinkedList {
3 private:
4     Node* tail; // tail->next is head
5 public:
6     CircularLinkedList() : tail(nullptr) {}
7     bool isEmpty() { return tail == nullptr; }
8
9     void addFront(int val) {
10         Node* newNode = new Node(val);
11         if (isEmpty()) {
12             tail = newNode;
13             tail->next = newNode; // back to itself
14         } else {
15             newNode->next = tail->next;
16             tail->next = newNode;
17         }
18     }
19
20     void addBack(int val) {
21         addFront(val);
22         tail = tail->next;
23     }
24
25     // Display the list
26     void display() {
27         if (isEmpty()) return;
28         Node* temp = tail->next; // start from head (tail->next)
29         do {
30             cout << temp->data << " -> ";
31             temp = temp->next;
32         } while (temp != tail->next);
33         cout << "(head)\n";
34     }
35
36
37     // The other function (Erase, Search, Insert, v.v.) is similar
38     // with SinglyLinkedList
39     // But be careful with tail.
40     void removeFront() {
41         if (isEmpty()) return;
42         if (tail->next == tail) { // Only one element
43             delete tail;
44             tail = nullptr;
45         } else {
46             Node* temp = tail->next;
47             tail->next = temp->next;
48             delete temp;

```



```

48         }
49     }
50 };

```

6.2.3 Danh sách liên kết kép

```

1 struct DNode {
2     int data;
3     DNode* next;
4     DNode* prev;
5     DNode(int val) : data(val), next(nullptr), prev(nullptr) {}
6 };
7 class DoublyLinkedList {
8 private:
9     DNode* head;
10 public:
11     DoublyLinkedList() : head(nullptr) {}
12     bool isEmpty() { return head == nullptr; }
13     void addFront(int val) {
14         DNode* newNode = new DNode(val);
15         newNode->next = head;
16         if (head) head->prev = newNode;
17         head = newNode;
18     }
19     void addBack(int val) {
20         DNode* newNode = new DNode(val);
21         if (isEmpty()) {
22             head = newNode;
23             return;
24         }
25         DNode* temp = head;
26         while (temp->next) temp = temp->next;
27         temp->next = newNode;
28         newNode->prev = temp;
29     }
30     void display() {
31         DNode* temp = head;
32         while (temp) {
33             cout << temp->data << " <-> ";
34             temp = temp->next;
35         }
36         cout << "NULL\n";
37     }
38     // Erase, Insert, Search is similar
39 };

```

7 CTDL Trừu tượng

7.1 Queue (Hàng đợi)

Queue là một trong những cấu trúc dữ liệu cơ bản và rất thường gặp trong lập trình cũng như các bài toán thực tế. Queue tuân theo nguyên tắc "vào trước ra trước" (FIFO),

nghĩa là phần tử nào được thêm vào trước sẽ được lấy ra trước.

* **Khái niệm:**

- Là cấu trúc dữ liệu FIFO (First In First Out): phần tử nào vào trước thì ra trước.
- Các phép toán chính:
- **Enqueue** (thêm vào cuối)
- **Dequeue** (lấy ra ở đầu)
- **Peek** (xem phần tử đầu, không xóa)

* **Chức năng:**

- Quản lý các phần tử theo thứ tự đến, phục vụ theo thứ tự đó.

7.1.1 Khi nào sử dụng queue?

Queue thường được dùng trong các tình huống cần xử lý theo thứ tự đến, ví dụ như lập lịch tiến trình, xử lý yêu cầu trong hệ thống máy in, hàng chờ giao dịch tại ngân hàng, hoặc xử lý các tác vụ theo thứ tự thời gian.

7.1.2 Độ phức tạp của queue

Độ phức tạp của các thao tác trên queue phụ thuộc vào cách cài đặt:

- * **Tốt nhất:** $O(1)$ (enqueue, dequeue, peek) khi sử dụng danh sách liên kết.
- * **Tệ nhất:** $O(n)$ (dequeue) nếu dùng mảng tĩnh và phải dịch chuyển các phần tử khi lấy ra.

7.1.3 Code minh họa queue bằng danh sách liên kết (C++)

```
1 #include <iostream>
2 using namespace std;
3
4 struct Node {
5     int data;
6     Node* next;
7     Node(int d) : data(d), next(nullptr) {}
8 };
9
10 class Queue {
11 private:
12     Node *front, *rear;
13 public:
14     Queue() : front(nullptr), rear(nullptr) {}
15     void enqueue(int x) {
16         Node* temp = new Node(x);
17         if (rear == nullptr) {
18             front = rear = temp;
19             return;
20         }
21         rear->next = temp;
22         rear = temp;
```

```

23     }
24     void dequeue() {
25         if (front == nullptr) return;
26         Node* temp = front;
27         front = front->next;
28         if (front == nullptr) rear = nullptr;
29         delete temp;
30     }
31     int peek() {
32         if (front == nullptr) return -1;
33         return front->data;
34     }
35 };
36
37 int main() {
38     Queue q;
39     q.enqueue(10);
40     q.enqueue(20);
41     q.enqueue(30);
42     cout << endl; // 10
43     q.dequeue();
44     cout << q.peek() << endl; // 20
45     return 0;
46 }

```

Listing 9: Minh họa queue bằng danh sách liên kết

7.2 Stack (Ngăn xếp)

Stack là một cấu trúc dữ liệu quan trọng khác, tuân theo nguyên tắc "vào sau ra trước" (LIFO). Stack thường được dùng để lưu trữ tạm thời các phần tử và xử lý theo thứ tự ngược lại với thứ tự đưa vào.

* Khái niệm:

- Là cấu trúc dữ liệu LIFO (Last In First Out): phần tử nào vào sau thì ra trước.
- Các phép toán chính:
- **Push** (thêm vào đỉnh)
- **Pop** (lấy ra ở đỉnh)
- **Peek** (xem phần tử đỉnh, không xóa)

* Chức năng:

- Quản lý các phần tử theo thứ tự vào ra ngược nhau.

7.2.1 Khi nào sử dụng stack?

Stack thường được sử dụng trong các tình huống như lưu trữ lịch sử các thao tác (undo/redo trong phần mềm), duyệt cây, chuyển đổi biểu thức toán học (infix sang postfix hoặc prefix), quản lý bộ nhớ và xử lý gọi hàm trong chương trình.

7.2.2 Độ phức tạp của stack

Độ phức tạp của các thao tác trên stack cũng phụ thuộc vào cách cài đặt:

- * **Tốt nhất:** $O(1)$ (push, pop, peek) khi sử dụng danh sách liên kết.
- * **Tệ nhất:** $O(n)$ (push) nếu dùng mảng động và phải mở rộng mảng khi đầy, nhưng thông thường vẫn là $O(1)$.

7.2.3 Code minh họa stack bằng danh sách liên kết (C++)

```
1  #include <iostream>
2  using namespace std;
3
4  struct Node {
5      int data;
6      Node* next;
7      Node(int d) : data(d), next(nullptr) {}
8  };
9
10 class Stack {
11 private:
12     Node* top;
13 public:
14     Stack() : top(nullptr) {}
15     void push(int x) {
16         Node* temp = new Node(x);
17         temp->next = top;
18         top = temp;
19     }
20     void pop() {
21         if (top == nullptr) return;
22         Node* temp = top;
23         top = top->next;
24         delete temp;
25     }
26     int peek() {
27         if (top == nullptr) return -1;
28         return top->data;
29     }
30 };
31
32 int main() {
33     Stack s;
34     s.push(10);
35     s.push(20);
36     s.push(30);
37     cout << s.peek() << endl; // 30
38     s.pop();
39     cout << s.peek() << endl; // 20
40     return 0;
41 }
```

Listing 10: Minh họa stack bằng danh sách liên kết

7.3 Priority Queue (Hàng đợi ưu tiên)

Priority queue là một cấu trúc dữ liệu nâng cao, cho phép mỗi phần tử có mức độ ưu tiên riêng. Phần tử nào có độ ưu tiên cao hơn sẽ được lấy ra trước, bất kể thứ tự đưa vào.

* Khái niệm:

- Là cấu trúc dữ liệu mà mỗi phần tử có mức độ ưu tiên, phần tử nào có độ ưu tiên cao hơn sẽ được lấy ra trước.
- Các phép toán chính:
 - **Insert** (thêm phần tử với độ ưu tiên)
 - **Extract** (lấy phần tử có độ ưu tiên cao nhất)
 - **Peek** (xem phần tử ưu tiên cao nhất, không xóa)

* Chức năng:

- Quản lý các phần tử theo độ ưu tiên, phục vụ phần tử ưu tiên cao nhất trước.

7.3.1 Khi nào sử dụng priority queue?

Priority queue thường được sử dụng trong các bài toán lập lịch tiến trình theo độ ưu tiên (hệ điều hành, router mạng), các thuật toán tìm đường ngắn nhất (Dijkstra, A*), hoặc trong các thuật toán sắp xếp như heap sort.

7.3.2 Độ phức tạp của priority queue

Độ phức tạp của priority queue phụ thuộc vào cách cài đặt:

* Tốt nhất:

- Nếu dùng heap: $O(1)$ (peek), $O(\log n)$ (insert, extract)
- Nếu dùng danh sách liên kết có thứ tự: $O(n)$ (insert), $O(1)$ (extract, peek)

* Tệ nhất:

- Nếu dùng danh sách liên kết không thứ tự: $O(n)$ (insert, extract, peek)

7.3.3 Code minh họa priority queue bằng danh sách liên kết (C++)

```
1 #include <iostream>
2 using namespace std;
3
4 struct Node {
5     int data;
6     int priority;
7     Node* next;
```

```

8     Node(int d, int p) : data(d), priority(p), next(nullptr) {}
9 };
10
11 class PriorityQueue {
12 private:
13     Node* head;
14 public:
15     PriorityQueue() : head(nullptr) {}
16
17     void insert(int x, int p) {
18         Node* temp = new Node(x, p);
19         temp->next = head;
20         head = temp;
21     }
22
23     void pop() {
24         if (head == nullptr) return;
25         Node *prev = nullptr, *curr = head;
26         Node *maxPrev = nullptr, *maxNode = head;
27         while (curr != nullptr) {
28             if (curr->priority > maxNode->priority) {
29                 maxNode = curr;
30                 maxPrev = prev;
31             }
32             prev = curr;
33             curr = curr->next;
34         }
35         if (maxPrev != nullptr) maxPrev->next = maxNode->next;
36         else head = maxNode->next;
37         delete maxNode;
38     }
39
40
41     int top() {
42         if (head == nullptr) return -1;
43         Node* curr = head;
44         Node* maxNode = head;
45         while (curr != nullptr) {
46             if (curr->priority > maxNode->priority) {
47                 maxNode = curr;
48             }
49             curr = curr->next;
50         }
51         return maxNode->data;
52     }
53
54
55     void display() {
56         Node* curr = head;
57         cout << "Priority Queue: ";
58         while (curr != nullptr) {
59             cout << "(" << curr->data << ", " << curr->priority << ") ";
60             ;
61             curr = curr->next;
62         }
63         cout << endl;

```

```

63     }
64 };
65
66 int main() {
67     PriorityQueue pq;
68     pq.insert(10, 1);
69     pq.insert(20, 3);
70     pq.insert(30, 2);
71     pq.display(); // Priority Queue: (30,2) (20,3) (10,1)
72     cout << "Phan tu uu tien cao nhat: " << pq.top() << endl; // 20
73     pq.pop();
74     pq.display(); // Priority Queue: (30,2) (10,1)
75     return 0;
76 }

```

Listing 11: Minh họa priority queue bằng danh sách liên kết (không thứ tự)

8 CTDL cây

8.1 Khái niệm, các tính chất

8.2 Duyệt theo chiều sâu (Depth-First Traversal)

* **Pre-order (NLR):** Xử lý nút $\rightarrow$ trái $\rightarrow$ phải

```

1 void preorder(Node* root) {
2     if (root) {
3         cout << root->data << " ";
4         preorder(root->left);
5         preorder(root->right);
6     }
7 }

```

* **In-order (LNR):** Trái $\rightarrow$ xử lý nút $\rightarrow$ phải

```

1 void inorder(Node* root) {
2     if (root) {
3         inorder(root->left);
4         cout << root->data << " ";
5         inorder(root->right);
6     }
7 }

```

* **Post-order (LRN):** Trái $\rightarrow$ phải $\rightarrow$ xử lý nút

```

1 void postorder(Node* root) {
2     if (root) {
3         postorder(root->left);
4         postorder(root->right);
5         cout << root->data << " ";
6     }
7 }

```

8.3 Duyệt theo chiều rộng (Breadth-First Traversal / Level-order)

```
1 #include <queue>
2 void levelOrder(Node* root) {
3     if (!root) return;
4     queue<Node*> q;
5     q.push(root);
6     while (!q.empty()) {
7         Node* curr = q.front(); q.pop();
8         cout << curr->data << " ";
9         if (curr->left) q.push(curr->left);
10        if (curr->right) q.push(curr->right);
11    }
12 }
```

8.4 Một số kiểu duyệt biến thể

* **Reverse In-order (RNL):** phải $\rightarrow$ nút $\rightarrow$ trái

```
1 void reverseInorder(Node* root) {
2     if (root) {
3         reverseInorder(root->right);
4         cout << root->data << " ";
5         reverseInorder(root->left);
6     }
7 }
```

* **Duyệt có điều kiện:** chỉ in nút có giá trị $> k$

```
1 void inorderGreater(Node* root, int k) {
2     if (root) {
3         inorderGreater(root->left, k);
4         if (root->data > k)
5             cout << root->data << " ";
6         inorderGreater(root->right, k);
7     }
8 }
```

8.5 Các loại cây thường gặp

8.5.1 BST

Cây nhị phân tìm kiếm (BST)

Cây nhị phân tìm kiếm (Binary Search Tree) là cấu trúc dữ liệu cây nhị phân, thỏa mãn:

- * Các khóa trong cây con trái $<$ khóa của gốc $<$ các khóa trong cây con phải
- * Cây con trái và phải đều là BST
- * Khóa duy nhất trong cây

Độ cao h của cây có N nút:

$$\log N \leq h \leq N$$

Trường hợp "đẹp nhất": $h \approx \log N$

Trường hợp suy biến (cây chỉ có 1 nhánh): $h \approx N$

Kỹ thuật lập trình thích hợp: đệ qui

Phép quay là thao tác cấu trúc lại cây nhị phân để giữ cho cây không bị nghiêng về một phía quá mức.

Quay trái: cây nghiêng phải $\rightarrow$ ta "đẩy" con phải lên, kéo cha xuống trái.

Quay phải: cây nghiêng trái $\rightarrow$ ta "đẩy" con trái lên, kéo cha xuống phải.

8.6 Các biến thể của BST

* Cây nhị phân tìm kiếm cân bằng hoàn toàn (CBT)

Là BST mà cây con trái và phải đều là CBT

Số nút của hai cây con lệch nhau không quá 1

Chiều cao $h_{CBT} = \lfloor \log_2 N \rfloor$

* Cây AVL

Là BST có cây con trái và phải đều là AVL

Độ cao của 2 cây con lệch nhau không quá 1

Chiều cao $h_{AVL} < 1.45 \times h_{CBT}$

* Cây đỏ đen (Red-Black Tree)

Là BST có:

- Các cạnh được tô màu đỏ hoặc đen
- Không có hai cạnh đỏ kề nhau trên đường đi từ gốc đến lá
- Cạnh nối với nút ngoài (NULL) luôn có màu đen
- Các đường đi từ gốc đến nút ngoài đều có số cạnh đen bằng nhau (chiều cao đen B)
- $\log_2(N + 1) \leq H \leq 2 \times \log_2(N + 1)$

Các thao tác cơ bản trên BST

```
1 struct Node {
2     int data;
3     Node* left;
4     Node* right;
5     Node(int val) : data(val), left(nullptr), right(nullptr) {}
6 };
7 Node* root = nullptr; // Empty tree
8 bool isEmpty(Node* node) { return node == nullptr; }
9
10 // Search
11 Node* search(Node* node, int key) {
12     if (!node || node->data == key) return node;
13     if (key < node->data)
14         return search(node->left, key);
```

```

15         else
16             return search(node->right, key);
17     }
18
19     // Insert
20     Node* insert(Node* node, int key) {
21         if (!node) return new Node(key);
22         if (key < node->data)
23             node->left = insert(node->left, key);
24         else if (key > node->data)
25             node->right = insert(node->right, key);
26         return node;
27     }
28
29     // Delete
30     Node* findMin(Node* node) {
31         while (node->left) node = node->left;
32         return node;
33     }
34     Node* remove(Node* node, int key) {
35         if (!node) return node;
36         if (key < node->data)
37             node->left = remove(node->left, key);
38         else if (key > node->data)
39             node->right = remove(node->right, key);
40         else {
41             // Node has less than two children
42             if (!node->left) {
43                 Node* tmp = node->right;
44                 delete node;
45                 return tmp;
46             }
47             else if (!node->right) {
48                 Node* tmp = node->left;
49                 delete node;
50                 return tmp;
51             }
52             // Node has two children: replace with successor
53             Node* tmp = findMin(node->right);
54             node->data = tmp->data;
55             node->right = remove(node->right, tmp->data);
56         }
57         return node;
58     }
59
60     // Traversal
61     void preorder(Node* node) {
62         if (node) {
63             cout << node->data << " ";
64             preorder(node->left);
65             preorder(node->right);
66         }
67     }
68     void inorder(Node* node) {
69         if (node) {
70             inorder(node->left);

```

```

71         cout << node->data << " ";
72         inorder(node->right);
73     }
74 }
75 void postorder(Node* node) {
76     if (node) {
77         postorder(node->left);
78         postorder(node->right);
79         cout << node->data << " ";
80     }
81 }
82
83 // Clear tree (release memory)
84 void clear(Node* &node) {
85     if (node) {
86         clear(node->left);
87         clear(node->right);
88         delete node;
89         node = nullptr;
90     }
91 }

```

Cách cài đặt BST

* Dùng con trỏ

Mỗi nút có 2 con trỏ: left, right

Linh hoạt, dễ mở rộng, dễ thay đổi cấu trúc

* Dùng mảng một chiều

Mỗi nút lưu 2 chỉ số nguyên là chỉ số cây con

Cần biến phụ quản lý chỉ số thêm mới

Ít linh hoạt hơn con trỏ

* Thư viện C++

`std::set`

`std::map`

Các cấu trúc này thường cài đặt bằng cây đỏ đen hoặc AVL, hỗ trợ sẵn thao tác thêm, xóa, tìm kiếm.

Tóm tắt điểm nổi bật

- * Khóa trong cây con trái < khóa của gốc < cây con phải
- * Cây con trái và phải đều là BST
- * Khóa duy nhất trong cây
- * Hiệu quả: tìm kiếm, thêm, xóa tốt ($O(h)$); trong trường hợp cân bằng: $h \approx \log N$
- * Trường hợp xấu nhất (cây suy biến): $h \approx N \rightarrow$ thao tác chậm
- * Nếu ưu tiên tìm kiếm nhanh và ít thay đổi dữ liệu, cây AVL có thể là lựa chọn tốt hơn. Nếu ưu tiên chèn và xóa nhanh, đặc biệt khi có nhiều thay đổi dữ liệu, cây đỏ đen có thể là lựa chọn tốt hơn.

8.7 B-Tree

B-Tree

B-Tree là cấu trúc dữ liệu cây cân bằng tổng quát, mỗi nút chứa nhiều khóa và nhiều cây con, thích hợp lưu trữ dữ liệu lớn trên đĩa.

Ứng dụng quan trọng: cây chỉ mục trong hệ quản trị cơ sở dữ liệu (DBMS).

8.7.1 Các biến thể của B-Tree

- * **B+ Tree**

Mọi dữ liệu chỉ lưu ở nút lá

Nút trong chỉ lưu khóa để định hướng tìm kiếm

- * **B* Tree**

Cân bằng tốt hơn, khi nút đầy thường chia sẻ với nút anh em trước khi tách nút

8.8 Các thao tác cơ bản

```
1 struct BTreeNode {
2     int *keys;           // Array to store keys
3     int t;               // Minimum degree
4     BTreeNode **C;       // Array of child pointers
5     int n;               // Current number of keys
6     bool leaf;           // True if leaf node
7     BTreeNode(int _t, bool _leaf) {
8         t = _t;
9         leaf = _leaf;
10        keys = new int[2*t-1];
11        C = new BTreeNode*[2*t];
12        n = 0;
13    }
14 };
15 // Empty tree: root = nullptr
16 BTreeNode* root = nullptr;
17 bool isEmpty() { return root == nullptr; }
18
19 // Search
20 BTreeNode* search(BTreeNode* node, int k) {
21     int i = 0;
22     while (i < node->n && k > node->keys[i])
23         i++;
24     if (i < node->n && node->keys[i] == k)
25         return node;
26     if (node->leaf)
27         return nullptr;
28     return search(node->C[i], k);
29 }
30
31 // Insert
32 void insert(int k) {
33     if (root == nullptr) {
34         root = new BTreeNode(t, true);
35         root->keys[0] = k;
```

```

36         root->n = 1;
37     } else {
38         // If root is full, split root (details not shown here)
39         // If not, insert into appropriate node
40     }
41 }
42
43 // Delete
44 // Deletion is complex (merge, borrow from siblings...), only outline
45 // is shown:
46 // If deletion is in a leaf: delete directly
47 // If deletion is in an internal node:
48 // If left child has >= t keys: replace with predecessor
49 // Or if right child has >= t keys: replace with successor
50 // If both have < t keys: merge into one node then delete
51
52 // Traverse
53 void traverse(BTreeNode* node) {
54     if (node) {
55         int i;
56         for (i = 0; i < node->n; i++) {
57             if (!node->leaf)
58                 traverse(node->C[i]);
59             cout << node->keys[i] << " ";
60         }
61         if (!node->leaf)
62             traverse(node->C[i]);
63     }
64 }
65
66 // Clear tree (release memory)
67 void clear(BTreeNode* &node) {
68     if (node) {
69         if (!node->leaf) {
70             for (int i = 0; i <= node->n; i++)
71                 clear(node->C[i]);
72         }
73         delete[] node->keys;
74         delete[] node->C;
75         delete node;
76         node = nullptr;
77     }
78 }

```

8.8.1 Cách cài đặt

Kết hợp:

- * Mảng một chiều để lưu khóa
- * Mảng con trỏ để lưu cây con
- * Giúp giảm số lần đọc đĩa (mỗi nút lưu nhiều khóa).

8.8.2 Các khái niệm cơ bản

* B-Tree bậc m:

- Mỗi nút có tối đa m cây con
- Nút trong có ít nhất $\lceil m/2 \rceil$ cây con
- Nút gốc (nếu không phải lá): ít nhất 2 cây con
- Nút có k cây con sẽ chứa k-1 khóa
- Các khóa trong cùng một nút sắp xếp tăng dần
- Tất cả nút lá đều ở cùng mức h-1

* Nút gốc, nút trong, nút lá:

- Nút gốc: cấp cao nhất, không có cha
- Nút trong: có cả cây con và cha
- Nút lá: không có cây con

* Mức của nút:

- Mức nút gốc thường tính là 0 hoặc 1 (tùy quy ước)
- Mỗi nút con có mức bằng mức của nút cha + 1

* Độ cao của B-Tree bậc m có n khóa:

$$h \leq \log_{\lceil m/2 \rceil} \left(\frac{n+1}{2} \right)$$

8.8.3 Điểm nổi bật của B-Tree

- * Lưu trữ dữ liệu rất lớn trên đĩa
- * Cân bằng (chiều cao thấp)
- * Tìm kiếm nhanh ($O(\log n)$)
- * Tổn chi phí cân bằng khi thêm/xoá
- * Đọc đĩa theo trang, giảm số lần truy xuất đĩa

8.8.4 Ứng dụng

- * Cây chỉ mục trong hệ quản trị CSDL
- * Quản lý tập tin, chỉ mục thứ cấp

9 Đồ thị

9.1 Các khái niệm cơ bản

1. **Đơn đồ thị vô hướng:**

Đơn đồ thị vô hướng $G = \langle V, E \rangle$, gồm V là tập các đỉnh, E là tập các cặp không có thứ tự gồm hai phần tử khác nhau của V gọi là các cạnh.

2. **Đa đồ thị vô hướng:**

Đa đồ thị vô hướng $G = \langle V, E \rangle$, gồm V là tập các đỉnh, E là họ các cặp không có thứ tự gồm hai phần tử khác nhau của V gọi là các cạnh. Hai cạnh e_1, e_2 được gọi là cạnh bội nếu chúng cùng tương ứng với một cặp đỉnh.

3. **Giả đồ thị vô hướng:**

Giả đồ thị vô hướng $G = \langle V, E \rangle$, gồm V là tập các đỉnh, E là họ các cặp không có thứ tự gồm hai phần tử không nhất thiết phải khác nhau của V gọi là các cạnh. Cạnh $e = (u, u)$ được gọi là khuyên.

4. **Đơn đồ thị có hướng:**

Đơn đồ thị có hướng $G = \langle V, E \rangle$, gồm V là tập các đỉnh, E là tập các cặp có thứ tự gồm hai phần tử của V gọi là các cung.

5. **Đa đồ thị có hướng:**

Đa đồ thị có hướng $G = \langle V, E \rangle$, gồm V là tập các đỉnh, E là họ các cặp có thứ tự gồm hai phần tử của V gọi là các cung. Hai cung e_1, e_2 tương ứng với cùng một cặp đỉnh được gọi là cung lặp.

6. **Liên thông:**

Đồ thị vô hướng được gọi là liên thông nếu luôn tìm được đường đi giữa hai đỉnh bất kỳ của đồ thị.

7. **Liên thông mạnh, liên thông yếu:**

Liên thông mạnh: Đồ thị có hướng gọi là liên thông mạnh nếu giữa hai đỉnh bất kỳ u, v luôn có đường đi từ u đến v .

Liên thông yếu: Đồ thị có hướng gọi là liên thông yếu nếu đồ thị vô hướng tương ứng của nó liên thông.

8. **Đỉnh trụ:**

Đỉnh trụ là đỉnh khi loại bỏ đỉnh này và các cạnh liên thuộc với đỉnh này sẽ làm tăng số thành phần liên thông của đồ thị.

9. **Cạnh cầu:**

Cạnh cầu là những cạnh khi loại bỏ nó (chỉ loại bỏ cạnh, không loại bỏ hai đỉnh) làm tăng số thành phần liên thông của đồ thị.

9.2 Biểu diễn đồ thị trên máy tính

* Danh sách kề (`vector<int> adj[]`)

* Ma trận kề (`int adj[N][N]`)

* Danh sách cạnh (`vector<Edge>`)

9.3 Các thuật toán duyệt đồ thị

9.3.1 BFS (Breadth-First Search)

```
1 void bfs(int start, vector<int> adj[], int n) {
2     vector<bool> visited(n, false);
3     queue<int> q;
4     visited[start] = true;
5     q.push(start);
6     while (!q.empty()) {
7         int u = q.front(); q.pop();
8         cout << u << " ";
9         for (int v : adj[u]) {
10             if (!visited[v]) {
11                 visited[v] = true;
12                 q.push(v);
13             }
14         }
15     }
16 }
```

9.3.2 DFS (Depth-First Search)

```
1 void dfs(int u, vector<int> adj[], vector<bool>& visited) {
2     visited[u] = true;
3     cout << u << " ";
4     for (int v : adj[u])
5         if (!visited[v])
6             dfs(v, adj, visited);
7 }
```

9.4 Bài toán đường đi

9.4.1 Dijkstra

* Thuật toán Dijkstra được sử dụng để tìm đường đi ngắn nhất từ 1 đỉnh tới mọi đỉnh còn lại trên đồ thị, có thể áp dụng cho cả đồ thị có hướng và vô hướng không chứa trọng số âm. Độ phức tạp: $O((E + V)\log V)$

1. Ta sử dụng hàng đợi ưu tiên Q lưu pair với first lưu khoảng cách đường đi ngắn nhất từ đỉnh nguồn, và second lưu đỉnh. Q sẽ sắp xếp các phần tử theo khoảng cách đường đi tăng dần, nếu 2 đỉnh có cùng khoảng cách đường đi thì sắp xếp theo số thứ tự đỉnh tăng dần.
2. Ban đầu Q chỉ bao gồm cặp $0, s$ với s là đỉnh bắt đầu của thuật toán. Thuật toán sẽ được lặp lại khi Q còn chưa rỗng, mỗi lần lặp sẽ lấy ra cặp (d, u) ở đỉnh hàng đợi Q tương ứng với đỉnh u là đỉnh có đường đi ngắn nhất tính từ đỉnh nguồn s . Nếu $d > \text{dist}[u]$ với $\text{dist}[u]$ là khoảng cách đường đi ngắn nhất ghi nhận được cho đỉnh u thì ta sẽ bỏ qua u .

3. Khi đỉnh u được xét nó sẽ cố gắng cập nhật khoảng cách ngắn nhất cho các đỉnh v kề với u , sau đó tiếp tục đẩy đỉnh v và $\text{dist}[v]$ vào hàng đợi, điều này có thể dẫn tới nhiều phiên bản khác nhau của cùng 1 đỉnh với khoảng cách khác nhau trong hàng đợi. Tuy nhiên ta sẽ luôn chọn phiên bản có khoảng cách ngắn nhất để xét trước.

```

1 void dijkstra(int start, vector<pair<int,int>> adj[], int n) {
2     vector<int> dist(n, INT_MAX);
3     dist[start] = 0;
4     priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq
5     ;
6     pq.push({0, start});
7     while (!pq.empty()) {
8         auto [d, u] = pq.top(); pq.pop();
9         if (d > dist[u]) continue;
10        for (auto [v, w] : adj[u])
11            if (dist[v] > dist[u] + w) {
12                dist[v] = dist[u] + w;
13                pq.push({dist[v], v});
14            }
15    }
16 }

```

9.4.2 Bellman-Ford

- * Thuật toán Bellman-Ford được sử dụng để tìm đường đi ngắn nhất từ 1 đỉnh tới mọi đỉnh còn lại trên đồ thị, khác với Dijkstra thì Bellman-Ford có thể áp dụng cho đồ thị với cạnh có trọng số âm, tuy nhiên không thể áp dụng nếu đồ thị có chu trình âm. Thuật toán này có thể sử dụng để xác định rằng đồ thị có chu trình âm. Độ phức tạp: $O(V * E)$.
- * Ban đầu ta sử dụng một mảng $d[]$ để lưu khoảng cách từ đỉnh nguồn s tới mọi đỉnh còn lại trên đồ thị, $d[s] = 0$ và $d[u] = \text{INF}$ (vô cùng lớn) với mọi đỉnh u còn lại trên đồ thị
Thuật toán lặp $n - 1$ bước, mỗi bước sẽ xét tất cả các cặp cạnh (u, v) có trọng số w . Nếu $d[v] > d[u] + w$ thì sẽ cập nhật $d[v]$.

```

1 void bellmanFord(int start, vector<tuple<int,int,int>>& edges, int n)
2 {
3     vector<int> dist(n, INT_MAX);
4     dist[start] = 0;
5     for (int i = 1; i < n; i++)
6         for (auto [u, v, w] : edges)
7             if (dist[u] != INT_MAX && dist[v] > dist[u]+w)
8                 dist[v] = dist[u]+w;
9     // Check negative cycle
10    for (auto [u, v, w] : edges)
11        if (dist[u] != INT201_MAX && dist[v] > dist[u]+w)
12            cout << "Negative cycle\n";
13 }

```

Đồ thị/Tính chất	BFS	Dijkstra	Bellman Ford
Độ phức tạp	$O(V + E)$	$O((V + E) \log V)$	$O(V * E)$
Max size	$V, E \leq 10^7$	$V, E \leq 3 \cdot 10^5$	$V \cdot E \leq 10^7$
Không trọng số	Tốt nhất	Tốt	Tệ
Có trọng số	WA	Tốt nhất	Tốt
Có trọng số âm	WA	Cần biến đổi	Tốt
Có chu trình âm	Không thể xác định	Không thể xác định	Có thể xác định
Đồ thị nhỏ	WA nếu có trọng số	Đúng nhưng không phù hợp	Đúng nhưng không phù hợp